



BUAP

Facultad de Ciencias de la Comunicación

Documentación del microservicio de logística

H. Puebla de Z. mayo 2026

Microservicio de Logística

Este proyecto es un microservicio portátil desarrollado en Node.js que gestiona y rastrea el ciclo de vida de los envíos de mercancía para la plataforma Urban Market. Cuenta con un frontend integrado para el rastreo de paquetes y está completamente contenedorizado con Docker.

Arquitectura del Proyecto

El sistema está estructurado bajo una arquitectura limpia basada en componentes separados:

- /public: Contiene la interfaz gráfica de usuario (Frontend) construida con HTML estático, estilos CSS y lógica de interacción en JavaScript (app.js).
- server.js: El punto de entrada de la aplicación que enciende el servidor HTTP.
- /routes (shipmentRoutes.js): El enrutador que intercepta las peticiones y define los endpoints de la API.
- /controllers (shipmentController.js): Contiene la lógica de negocio, validaciones de datos (códigos postales, peso) y asignación automática de paqueterías.
- /models (shipmentModel.js): Gestiona la persistencia de los datos de los envíos.

Guía de Despliegue Local (Docker)

Para ejecutar este proyecto en un entorno aislado sin necesidad de instalar dependencias locales de desarrollo, sigue estos pasos:

1. Construir la Imagen de Docker

Navega a la carpeta raíz del proyecto y compila la imagen ejecutando:

```
``bash docker build -t mi-pagina-web .
```

2. Encender el Contenedor

Levanta el servicio mapeando el puerto interno hacia un puerto libre en tu computadora (Nosotros usamos el 5000 para evitar choques con XAMPP):

```
docker run -d -p 5000:3000
```

3. Acceso a la Aplicación

- Interfaz Web (Rastreador): Abre tu navegador e ingresa a <http://localhost:5000>

Documentación de la API (Endpoints)

El microservicio expone los siguientes endpoints para interactuar con el sistema de mensajería:

Método	Endpoint	Descripción	Requisitos / Formato
GET	/shipments	Obtiene la lista de todos los envíos registrados.	Ninguno. Devuelve un arreglo JSON.
POST	/shipments	Crea y programa un nuevo envío en el sistema.	Requiere un JSON con datos válidos (ver abajo).
GET	/shipments/:id	Consulta los detalles y la ubicación de una guía específica.	Reemplazar :id por el número de tracking.
PUT	/shipments/:id/status	Actualiza el estado logístico o ubicación actual del paquete.	JSON con status y current_location
DELETE	/shipments/:id/cancel	Cancela permanentemente el flujo de un envío en preparación.	Cambia el estado a CANCELADO.

Estructura para registrar un envío (POST /shipments)

El cuerpo de la petición debe ser un objeto JSON estructurado de la siguiente manera:

```
{
  "orderId": "ORD-999",
  "recipientName": "Nombre Destinatario",
  "address": "Calle y Número 123",
  "zipCode": "72000 (codigo postal)",
  "city": "Puebla",
  "weight": 4.5 (peso)
}
```

Validaciones de seguridad internas: * El código postal (zipCode) debe tener estrictamente 5 dígitos.

- El peso (weight) debe ser un valor numérico mayor a 0.
- El sistema calcula automáticamente la paquetería ideal según el peso: DHL Express (\$≤\$ 5kg), Estafeta Premium (5kg a 10kg) o FedEx Heavy (\$>\$ 10kg).
- El número de rastreo se genera automáticamente bajo el formato aleatorio TRACK-XXXXXX.

Pruebas Manuales Rápidas

Puedes registrar datos de prueba directo desde la consola del Símbolo del Sistema (CMD) de Windows con el siguiente comando:

```
curl -X POST http://localhost:5000/shipments -H "Content-Type: application/json" -d
{"orderId":"ORD-100","recipientName":"Josue","address":"Av Central
123","zipCode":"72000","city":"Puebla","weight":4.5}
```

Flujo de Estados y Reglas de Negocio

El microservicio controla estrictamente las transiciones del estado de un envío mediante máquinas de estado internas en shipmentController.js. Esto evita modificaciones erróneas una vez que el paquete ha avanzado en su ciclo logístico.

Estados Permitidos

Los estados válidos por los que puede transitar un paquete en una actualización normal (PUT /shipments/:id/status) son:

- PREPARACION: El paquete se está empaquetando en el almacén.
- RECOLECCION: La paquetería (DHL, Estafeta, FedEx) está recogiendo el paquete.
- TRANSITO: El paquete viaja hacia la ciudad de destino.
- ENTREGADO: El usuario final recibió el producto (Estado terminal positivo).

Restricciones Críticas de Seguridad

- Estados Terminales Inmutables: Si un envío cambia a estado CANCELADO o DEVUELTO, el sistema bloqueará de forma permanente cualquier intento posterior de modificar su estado o ubicación, respondiendo con un error 400 Bad Request.
- Regla de Cancelación (DELETE): * Si el paquete está en PREPARACION, se cancela de forma inmediata en el sistema.
- Si el paquete ya pasó a RECOLECCION o TRANSITO, el sistema cambia el flujo automáticamente y arranca un proceso de retorno hacia el almacén de origen.
- Regla de Devolución (POST /return): No se puede solicitar la devolución de un paquete si este sigue en PREPARACION. Debe haber sido recolectado o entregado previamente.

Estructura de Datos (Esquema de Respuesta JSON)

Cuando consultas un envío de manera exitosa (`GET /shipments/:id`), el microservicio responde con un objeto JSON estructurado que no solo contiene la información del destinatario, sino también un historial detallado de eventos en tiempo real:

```
{
  "orderId": "ORD-100",
  "trackingNumber": "TRACK-416696",
  "courier": "DHL Express",
  "weight": 4.5,
  "recipientName": "Josue",
  "address": "Av Central 123",
  "zipCode": "72000", "city": "Puebla",
  "tracking_events": [ { "status": "PREPARACION",
  "location": "Almacén Central Puebla",
  "timestamp": "2026-05-24T02:39:06.000Z"
  }
]
```

Respuestas del Servidor y Manejo de Errores

El sistema utiliza códigos de estado HTTP estándar para comunicar el resultado de las operaciones:

- 200 OK: La consulta, actualización o cancelación se procesó correctamente.
- 201 Created: El envío fue registrado con éxito en el sistema.
- 400 Bad Request: Datos de entrada inválidos. Devuelve un JSON explicando la causa exacta:
 - `{"error": "Campos obligatorios incompletos"}`
 - `{"error": "El peso debe ser mayor a 0"}`
 - `{"error": "El código postal debe tener exactamente 5 dígitos"}`
- 404 Not Found: El número de guía consultado no existe en el sistema o la ruta web es incorrecta.
- 500 Internal Server Error: Ocurrió un fallo inesperado en el servidor o se perdió la comunicación con el módulo de persistencia de datos.

Pasos para Configurar el Acceso Público:

1. Autenticación (Solo la primera vez):

Para evitar bloqueos de seguridad, inicia sesión en el panel de Ngrok, copia tu credencial de acceso (`AuthToken`) y regístrala en tu terminal ejecutando:

`ngrok config add-authtoken` (Tu token de seguridad)

2. Abrir el Túnel de Comunicación: Con el contenedor de Docker corriendo activamente en el puerto 5000, abre una ventana del Símbolo del Sistema (CMD) y levanta el puente de red:

`ngrok http 5000`

3. Compartir el Enlace Seguro:

- En la interfaz de la consola de Ngrok, localiza la línea que comienza con Forwarding.
- Copia la URL pública generada (ej. <https://a1b2-cd34.ngrok-free.app>).
- Cualquier petición externa dirigida a ese enlace será redirigida automáticamente hacia el contenedor de Docker en tu máquina local.

Nota: El enlace permanecerá activo de manera ilimitada mientras la ventana de la terminal de Ngrok continúe abierta. Si la terminal se cierra o el proceso se interrumpe, el enlace se destruirá de inmediato por seguridad y la próxima ejecución generará una URL completamente nueva.